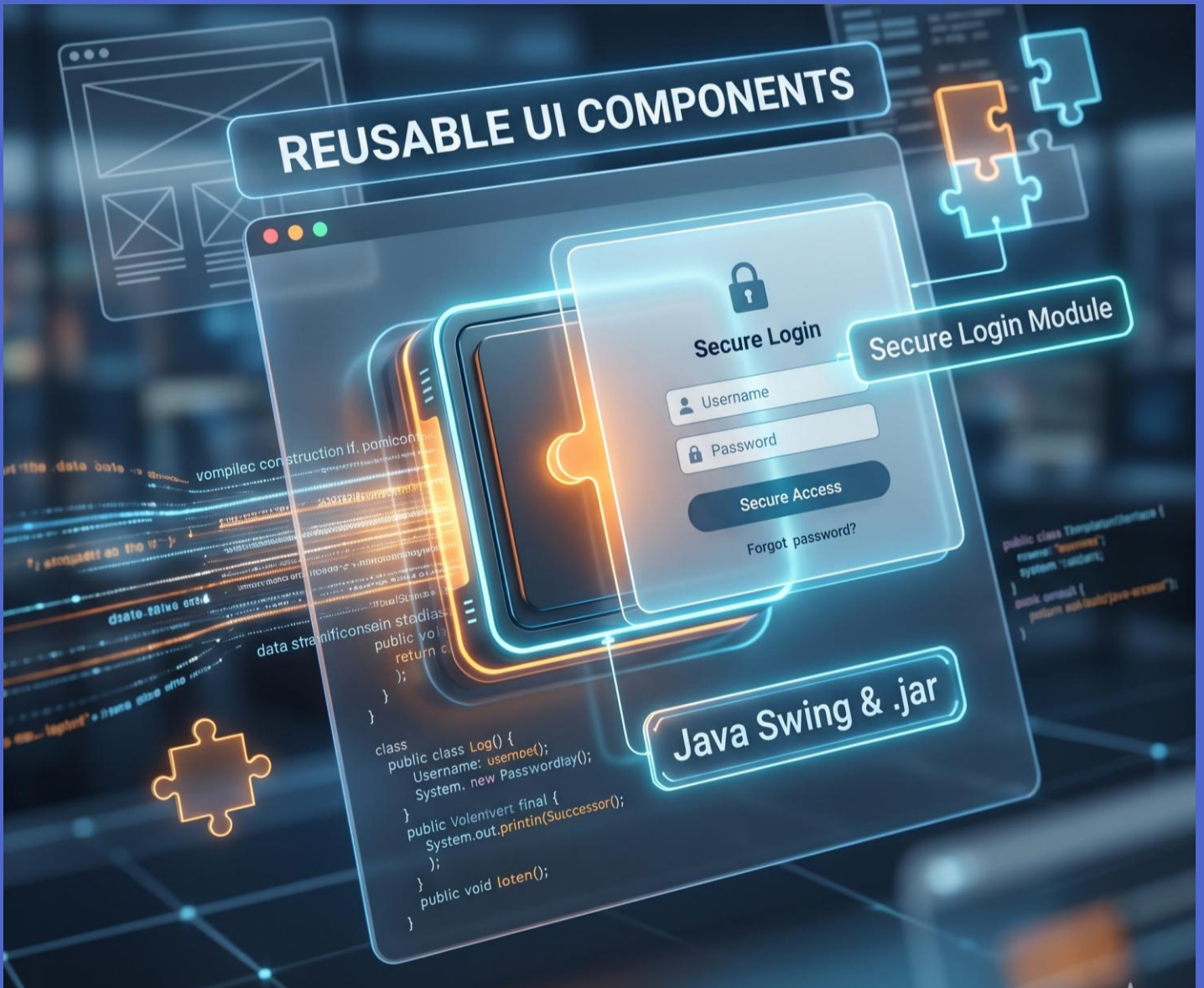


ACCESO A DATOS

CASO PRACTICO I UD6



ALUMNO CESUR

25/26

Alejandro Muñoz de la Sierra

PROFESOR

Patricio Santiago Fernández Florez

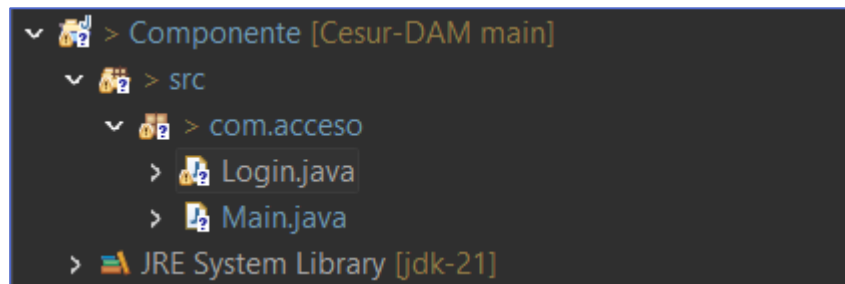
INTRODUCCION

A raíz del problema que nos planteó María, la responsable del área de desarrollo de la empresa, asumimos el reto de intentar estandarizar un poco la capa visual de nuestras aplicaciones internas. Nos pasaba mucho que, al arrancar un proyecto nuevo de gestión (como este de control de clientes y productos que tenemos entre manos), perdíamos un montón de tiempo volviendo a programar desde cero la típica ventana de login o los formularios básicos. Aparte de ser una tarea bastante redundante, al final cada compañero hacía la interfaz a su manera y perdíamos la coherencia gráfica entre las distintas herramientas de la empresa.

Para dar solución a esto, y aplicando lo que hemos visto en la Unidad 6 sobre programación orientada a componentes, hemos montado un componente visual empaquetado al que hemos llamado "Acceso al Sistema". La idea era tratar el código como si fuera un módulo independiente y reutilizable. Lo hemos desarrollado tirando a fondo de la librería Swing de Java, cumpliendo con los requisitos del enunciado pero intentando meterle un par de extras para mejorar un poco la experiencia de usuario (UX).

DISEÑO DEL FORMULARIO "ACCESO AL SISTEMA", COMPONENTES ADICIONALES Y RESOLUCIÓN DE CONFLICTOS

A la hora de diseñar el **formulario**, nuestra idea era buscar que fuera visualmente **limpio** y con una **funcionalidad estandarizada**. En vez de quedarnos en la típica ventana cutre con el usuario y la contraseña, quisimos hacer una interfaz un poco más moderna y que tuviera sentido en un entorno real de empresa.



1.1. Arquitectura de la Clase y el Constructor

El **diseño** lo hemos enfocado buscando que fuera **limpio** y sobre todo **funcional**. En vez de hacer la típica ventana sosa con dos cajas de texto, intentamos que pareciera algo más moderno y adaptado a un entorno de empresa.

Todo parte de la **clase Login**, que **hereda** directamente de **JFrame** para autoconstruir su propio lienzo **JFrame** (**public class Login extends JFrame**). Tomar esta decisión es clave porque convierte a nuestro **componente** en una **ventana gráfica** totalmente **independiente**. Así, cualquier otro desarrollador que importe el .jar solo tendrá que instanciar el objeto y ya tendrá toda la interfaz montada en pantalla.

```

Acceso a Datos - Componente/src/com/acceso/Login.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

catalogo-xml/pom.xml BaseXClient.java Gestionproductos.java AppLudoteca.java Login.java X Main.java

1 package com.acceso;
2
3 // Importamos AWT (Abstract Window Toolkit). Aunque usamos Swing para los botones y cajas,
4 // AWT sigue siendo el motor subyacente para manejar colores (Color), dimensiones (Dimension),
5 // fuentes tipográficas (Font) y alineaciones (Component).
6 import java.awt.Color;
7 import java.awt.Component;
8 import java.awt.Dimension;
9 import java.awt.Font;
10 import java.awt.GridLayout;
11 // Importamos los escuchadores de eventos de acción. Sin esto, nuestros botones
12 // serían simples dibujos en pantalla que no harían nada al hacer clic sobre ellos.
13 import java.awt.event.ActionEvent;
14 import java.awt.event.ActionListener;
15 // -----
16
17 // Importamos la librería Swing al completo. Swing es la evolución de AWT y nos
18 // proporciona componentes gráficos "ligeros" (lightweight) dibujados completamente en Java.
19 import javax.swing.Box;
20 import javax.swing.BoxLayout;
21 import javax.swing.JButton;
22 import javax.swing.JCheckBox;
23 import javax.swing.JComboBox;
24 import javax.swing.JFrame;
25 import javax.swing.JLabel;
26 import javax.swing.JOptionPane;
27 import javax.swing.JPanel;
28 import javax.swing.JPasswordField;
29 import javax.swing.JProgressBar;
30 import javax.swing.JTextField;
31 // Importamos EmptyBorder para poder aplicar márgenes invisibles (padding) a nuestros paneles.
32 // Esto es vital para que la interfaz "respire" y no se vea agobiante.
33 import javax.swing.border.EmptyBorder;
34 // Estos eventos específicos de documentos (DocumentEvent y DocumentListener) son necesarios
35 // para nuestro medidor de seguridad. Nos permiten "escuchar" cada pulsación de tecla
36 // en tiempo real dentro del campo de la contraseña.
37 import javax.swing.event.DocumentEvent;
38 import javax.swing.event.DocumentListener;
39

```

```

40 /**
41  * Componente visual reutilizable para el Acceso al Sistema. Decidimos extender
42  * de la clase JFrame para que este componente actúe como una ventana
43  * independiente y autosuficiente. De este modo, cualquier compañero del equipo
44  * de desarrollo solo tendrá que instanciar la clase para usarla.
45  */
46 public class Login extends JFrame {
47
48     // --- VARIABLES DE CLASE (COMPONENTES VISUALES) ---
49     // Agrupamos los elementos estándar solicitados por la rúbrica.
50     private JTextField txtUsuario;
51     private JPasswordField txtPassword; // Usamos JPasswordField para ocultar los caracteres por seguridad
52     private JButton btnEntrar;
53     private JButton btnRegistrar;
54
55     // Agrupamos nuestros 3 componentes adicionales que aportan el valor añadido
56     // (Innovación).
57     private JComboBox<String> comboRol; // 1. Selector de perfil de acceso
58     private JCheckBox chkMostrarPass; // 2. Control interactivo de visibilidad de contraseña
59     private JProgressBar progressSeguridad; // 3. Medidor visual de la fuerza de la contraseña
60

```

Durante los primeros pasos creando el código, nos pasó una cosa bastante curiosa con la sintaxis. Al ir a definir el constructor, por pura inercia escribimos la palabra reservada **class** dentro (**public class Login()**). Obviamente, Eclipse nos saltó al cuello enseguida con un **error de anidamiento** (*The nested type Login cannot hide an enclosing type*). La verdad es que el fallo nos vino genial para repasar algo **súper básico de Java** que a veces se te olvida: los **constructores tienen que llamarse exactamente igual que la clase, pero no llevan tipo de retorno ni palabras reservadas** en su definición. En cuanto lo dejamos como **public Login()**, el entorno ya reconoció bien el bloque de código donde íbamos a empezar a ensamblar la parte visual.

```
61● /**
62  * CONSTRUCTOR DEL COMPONENTE Aquí inicializamos y ensamblamos toda la interfaz
63  * gráfica en el momento en que se hace el 'new Login()'.
64  */
65● public Login() {
66    // 1. Configuración del marco principal (Ventana)
67    setTitle("Acceso al Sistema - Corporativo");
68    setSize(380, 420); // Dimensiones probadas para un diseño vertical equilibrado
69    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Finaliza el proceso en memoria al cerrar
70    setLocationRelativeTo(null); // Centra la ventana automáticamente en el monitor
71    setResizable(false); // Bloqueamos el redimensionamiento para evitar que se rompa el diseño
72
73    // 2. Creación del panel principal (El lienzo)
74    JPanel panelPrincipal = new JPanel();
75    // Usamos BoxLayout en el eje Y (vertical) para apilar los elementos de arriba a
76    // abajo
77    panelPrincipal.setLayout(new BoxLayout(panelPrincipal, BoxLayout.Y_AXIS));
78    // Aplicamos el padding (arriba, izquierda, abajo, derecha) para evitar que los
79    // elementos toquen los bordes
80    panelPrincipal.setBorder(new EmptyBorder(20, 25, 20, 25));
81    panelPrincipal.setBackground(new Color(245, 245, 250)); // Un gris muy claro, más moderno que el blanco puro
82
83    // 3. Título de bienvenida
84    JLabel lblTitulo = new JLabel("Bienvenido al Sistema");
85    lblTitulo.setFont(new Font("Segoe UI", Font.BOLD, 22)); // Tipografía corporativa y limpia
86    lblTitulo.setAlignmentX(Component.CENTER_ALIGNMENT);
87    lblTitulo.setForeground(new Color(40, 40, 60));
88
89    panelPrincipal.add(lblTitulo);
90    // Usamos RigidArea para crear separaciones físicas (márgenes) entre componentes
91    panelPrincipal.add(Box.createRigidArea(new Dimension(0, 25)));
92
```

1.2. Ensamblaje Visual y Componentes de Innovación

Todo el **componente principal** se encarga de **autoconstruir su propio panel** usando gestores de distribución geométricos. En nuestro caso, tiramos de un **BoxLayout** vertical para que los elementos se fueran apilando ordenadamente, y le metimos un **EmptyBorder** para que la interfaz tuviera márgenes y "respirara" un poco visualmente.

```

93 // --- SECCIÓN 1: NOMBRE DE USUARIO ---
94 JLabel lblUsuario = new JLabel("Nombre de Usuario:");
95 lblUsuario.setAlignmentX(Component.LEFT_ALIGNMENT); // Alineación a la izquierda
96
97 txtUsuario = new JTextField();
98 // Limitamos el alto de la caja de texto, pero dejamos que ocupe todo el ancho
99 // disponible
100 txtUsuario.setMaximumSize(new Dimension(Integer.MAX_VALUE, 30));
101
102 panelPrincipal.add(lblUsuario);
103 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 5)));
104 panelPrincipal.add(txtUsuario);
105 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 15)));
106
107 // --- SECCIÓN 2: CONTRASEÑA ---
108 JLabel lblPassword = new JLabel("Contraseña:");
109 lblPassword.setAlignmentX(Component.LEFT_ALIGNMENT);
110
111 txtPassword = new JPasswordField();
112 txtPassword.setMaximumSize(new Dimension(Integer.MAX_VALUE, 30));
113
114 panelPrincipal.add(lblPassword);
115 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 5)));
116 panelPrincipal.add(txtPassword);

```

Aparte de poner los **campos obligatorios** de toda la vida (el **JTextField** para el usuario y el **JPasswordField** para la clave), le dimos una vuelta a ver qué podíamos meter que de verdad aportara algo de innovación (y mejorara la UX). Al final, logramos meter estos tres extras:

- **Selector de Perfil (JComboBox<String>):** Como en cualquier empresa hay distintos **roles**, metimos un **desplegable** para que el usuario elija su perfil (Administrador, Empleado Estándar o Invitado). Viene muy bien para que el *backend* vaya precargando los permisos antes incluso de atacar a la base de datos para validar.

```

163 // [Componente Extra 3]: JComboBox (Lista desplegable de Roles)
164 JLabel lblRol = new JLabel("Perfil de Acceso:");
165 // Inicializamos el desplegable con un array de Strings predefinido
166 comboRol = new JComboBox<>(new String[] { "Administrador", "Empleado (Estándar)", "Invitado / Temporal" });
167 comboRol.setMaximumSize(new Dimension(Integer.MAX_VALUE, 30));
168
169 panelPrincipal.add(lblRol);
170 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 5)));
171 panelPrincipal.add(comboRol);
172 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 25)));
173

```


- **Control de Visibilidad de Clave (JCheckBox):** Siguiendo un poco lo que se hace hoy en día por **accesibilidad**, pusimos una casilla de "**Mostrar contraseña**". Al hacer clic, un **ActionListener** pilla el evento y usa el método **setEchoChar** para quitar o poner la ocultación del texto. Ayuda un montón a ver si has metido la pata al escribir.

```

118 // --- SECCIÓN 3: NUESTROS COMPONENTES ADICIONALES DE INNOVACIÓN ---
119
120 // [Componente Extra 1]: CheckBox para mostrar/ocultar contraseña
121 // Añade un control de accesibilidad muy valorado por los usuarios hoy en día.
122 chkMostrarPass = new JCheckBox("Mostrar contraseña");
123 chkMostrarPass.setBackground(new Color(245, 245, 250)); // Mismo fondo que el panel para que se integre
124 chkMostrarPass.setFont(new Font("Segoe UI", Font.PLAIN, 11));
125
126 // Añadimos lógica dinámica al CheckBox mediante una función Lambda
127 chkMostrarPass.addActionListener(e -> {
128     if (chkMostrarPass.isSelected()) {
129         txtPassword.setEchoChar((char) 0); // Al pasar el carácter 0 (nulo), mostramos el texto real
130     } else {
131         txtPassword.setEchoChar('*'); // Al desmarcarlo, volvemos a poner el punto de ocultación
132     }
133 });
134 panelPrincipal.add(chkMostrarPass);
135

```

- **Medidor Dinámico de Seguridad (JProgressBar):** Para nosotros, este es el componente estrella de la práctica. Enganchamos una **barra de progreso** de **Swing** al campo de la **contraseña** usando un **DocumentListener**. Según vas escribiendo, el sistema **evalúa** lo larga que es la clave **en tiempo real** y va llenando la barra (de 0 a 100), además de cambiarle el color de rojo a verde según la robustez.

```

136 // [Componente Extra 2]: Barra de progreso interactiva (Fuerza de contraseña)
137 progressSeguridad = new JProgressBar(0, 100);
138 progressSeguridad.setStringPainted(true); // Permitimos que la barra muestre texto superpuesto
139 progressSeguridad.setString("Fuerza de la contraseña");
140 progressSeguridad.setMaximumSize(new Dimension(Integer.MAX_VALUE, 15));
141
142 // Añadimos un "Listener" al documento interno del campo de contraseña.
143 // Así podemos reaccionar a cada tecla que pulsa o borra el usuario en tiempo
144 // real.
145 txtPassword.getDocument().addDocumentListener(new DocumentListener() {
146     public void insertUpdate(DocumentEvent e) {
147         evaluarPassword();
148     }
149
150     public void removeUpdate(DocumentEvent e) {
151         evaluarPassword();
152     }
153
154     public void changedUpdate(DocumentEvent e) {
155         evaluarPassword();
156     }
157 });
158
159 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 5)));
160 panelPrincipal.add(progressSeguridad);
161 panelPrincipal.add(Box.createRigidArea(new Dimension(0, 15)));
162

```

```

215  /**
216   * Método lógico auxiliar. Lee la longitud de la contraseña actual y actualiza
217   * el color y valor de la barra de progreso. Decidimos separarlo en un método
218   * privado para mantener el código limpio e instanciarlo fácilmente.
219   */
220  private void evaluarPassword() {
221      String pass = new String(txtPassword.getPassword()); // Extraemos la clave de forma segura
222
223      if (pass.length() == 0) {
224          progressSeguridad.setValue(0);
225          progressSeguridad.setForeground(Color.GRAY);
226      } else if (pass.length() < 5) {
227          progressSeguridad.setValue(30);
228          progressSeguridad.setForeground(Color.RED); // Insegura
229      } else if (pass.length() < 8) {
230          progressSeguridad.setValue(60);
231          progressSeguridad.setForeground(Color.ORANGE); // Nivel medio
232      } else {
233          progressSeguridad.setValue(100);
234          progressSeguridad.setForeground(new Color(40, 167, 69)); // Segura (Verde)
235      }
236  }
237 }

```

1.3 Implementación de la funcionalidad del botón "Registrar"

```

174  // --- SECCIÓN 4: BOTONERA ---
175  // Usamos un nuevo panel secundario con GridLayout (1 fila, 2 columnas)
176  // para que ambos botones midan exactamente lo mismo.
177  JPanel panelBotones = new JPanel(new GridLayout(1, 2, 10, 0));
178  panelBotones.setBackground(new Color(245, 245, 250));
179
180  btnEntrar = new JButton("Iniciar Sesión");
181  btnEntrar.setBackground(new Color(60, 120, 216)); // Azul corporativo
182  btnEntrar.setForeground(Color.WHITE);
183  btnEntrar.setFocusPainted(false); // Eliminamos el feo borde punteado nativo de Java al hacer clic
184
185  btnRegistrar = new JButton("Registrar");
186  btnRegistrar.setBackground(new Color(40, 167, 69)); // Verde para indicar creación/alta
187  btnRegistrar.setForeground(Color.WHITE);
188  btnRegistrar.setFocusPainted(false);
189

```

Tal y como pedía la rúbrica al pie de la letra, le dimos **funcionalidad** al **botón** dedicado a los **nuevos usuarios (JButton)**. Para ello le registramos un **ActionListener** normal y corriente que se queda a la escucha de clics. Cuando el **usuario pulsa** el botón, el programa frena el flujo principal y **lanza** un **cuadro de diálogo** tirando de la clase **JOptionPane** estándar de Swing. Lo dejamos configurado para que salte exactamente con el mensaje que se exigía: **"Ha pulsado el botón Registrar"**. Es una ejecución casi instantánea que hace su función y no genera ningún conflicto con las validaciones del resto de los campos.

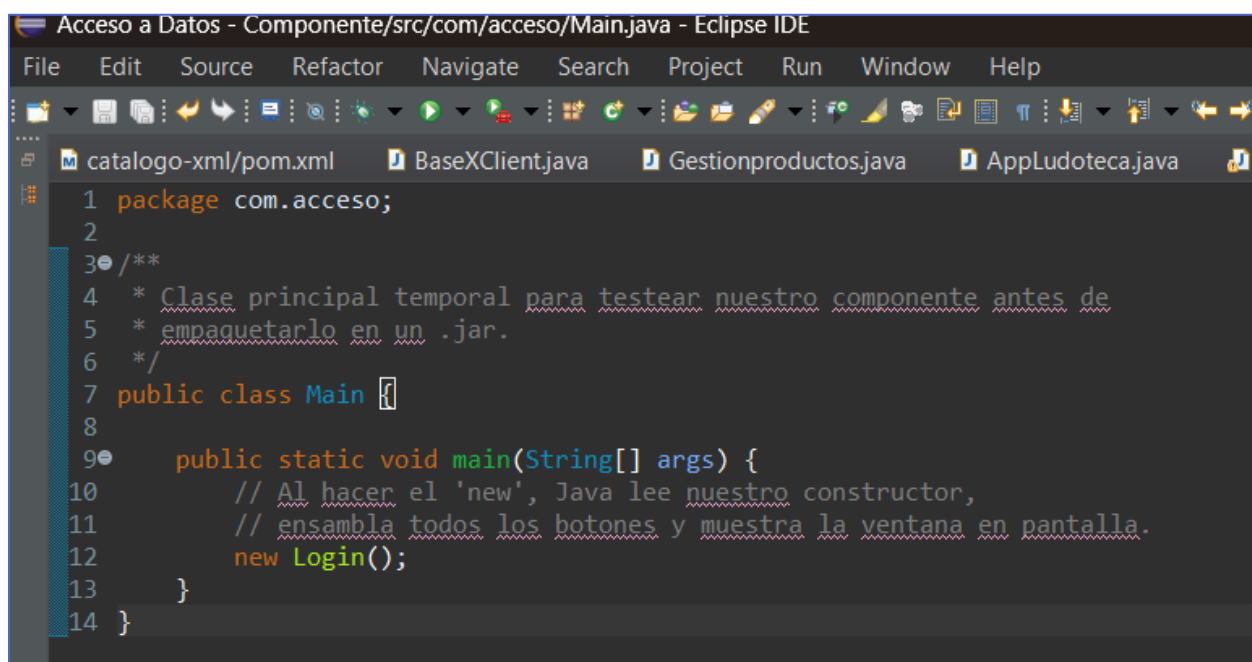

```

190 // --- REQUISITO ESPECÍFICO DEL CASO PRÁCTICO ---
191 // El botón Registrar debe mostrar un mensaje concreto. Lo implementamos aquí.
192 btnRegistrar.addActionListener(new ActionListener() {
193     @Override
194     public void actionPerformed(ActionEvent e) {
195         // Invocamos el cuadro de diálogo estándar de información
196         JOptionPane.showMessageDialog(Login.this, "Ha pulsado el botón Registrar", // Mensaje exacto requerido
197             "Información de Registro", // Título de la ventanita
198             JOptionPane.INFORMATION_MESSAGE); // Icono de información (la 'i' azul)
199     }
200 });
201
202 // Ensamblamos los botones en su panel secundario
203 panelBotones.add(btnEntrar);
204 panelBotones.add(btnRegistrar);
205
206 // Añadimos la botonera al panel vertical principal
207 panelPrincipal.add(panelBotones);
208
209 // Finalmente, añadimos todo el lienzo ensamblado a nuestra ventana JFrame y la
210 // hacemos visible
211 add(panelPrincipal);
212 setVisible(true);
213 }

```

1.4. Pruebas Locales y Resolución del Conflicto de Compilación

Antes de empaquetar la clase en un archivo .jar, teníamos que **probar** que la **vista** se renderizaba bien. Aquí aplicamos la regla de oro de crear componentes: nuestra clase **Login** **no** podía tener un método **main**, porque **es** una **librería** que se va a usar en otros sitios, no una app autoejecutable. Para poder probarlo, **hicimos** una **clase auxiliar** de "usar y tirar" llamada **Main.java** dentro del mismo proyecto, que lo único que hacía era instanciar el componente (new Login();) dentro de su main.



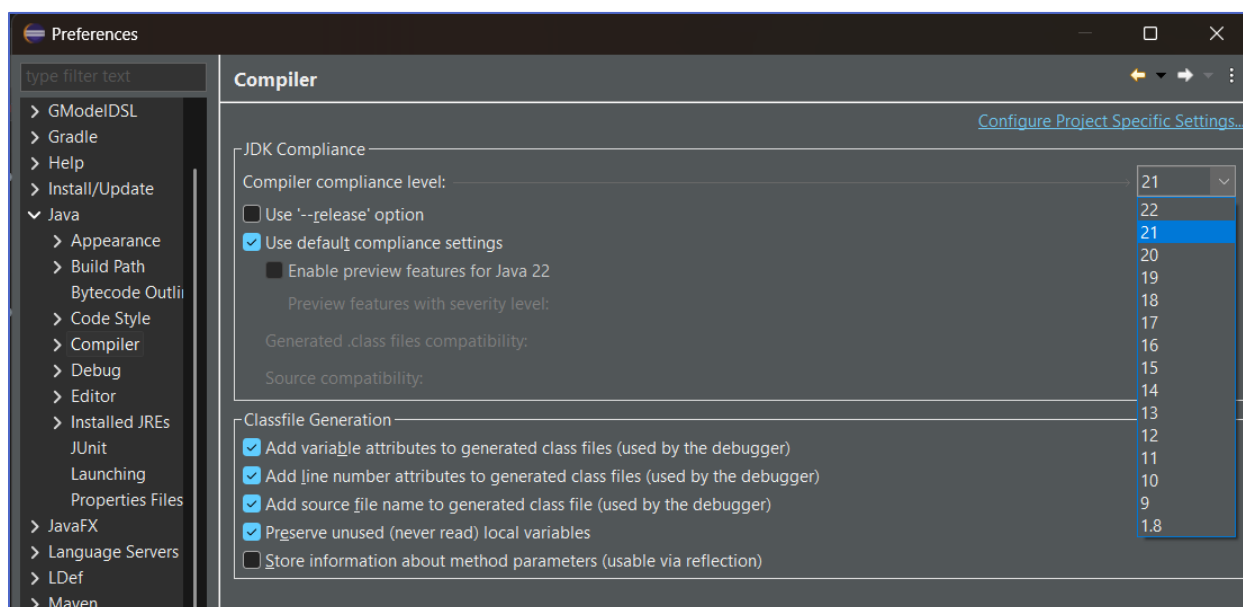
```

Acceso a Datos - Componente/src/com/acceso/Main.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
catalogo-xml/pom.xml BaseXClient.java Gestionproductos.java AppLudoteca.java
1 package com.acceso;
2
3 /**
4  * Clase principal temporal para testear nuestro componente antes de
5  * empaquetarlo en un .jar.
6  */
7 public class Main {
8
9     public static void main(String[] args) {
10         // Al hacer el 'new', Java lee nuestro constructor,
11         // ensambla todos los botones y muestra la ventana en pantalla.
12         new Login();
13     }
14 }

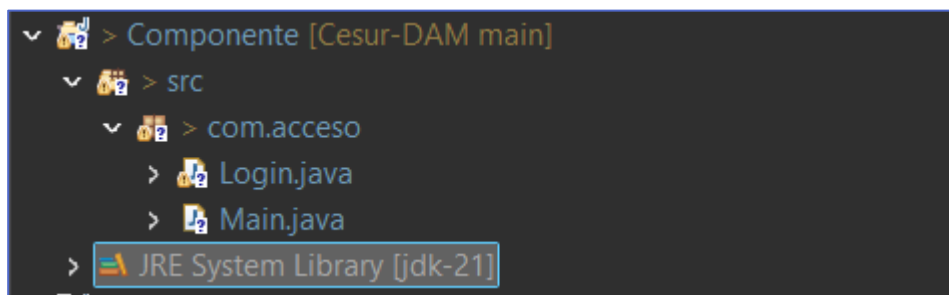
```

El problema fue que, al darle al **play** por primera vez, nos saltó un **error** crítico que tumbó la máquina virtual de Java: un **UnsupportedClassVersionError**. Mirando la traza del error, vimos que era un problema de desincronización de versiones (el aviso decía que el archivo se había **compilado** en la **versión 66.0**, pero el **entorno** solo **ejecuta** hasta la **65.0**).

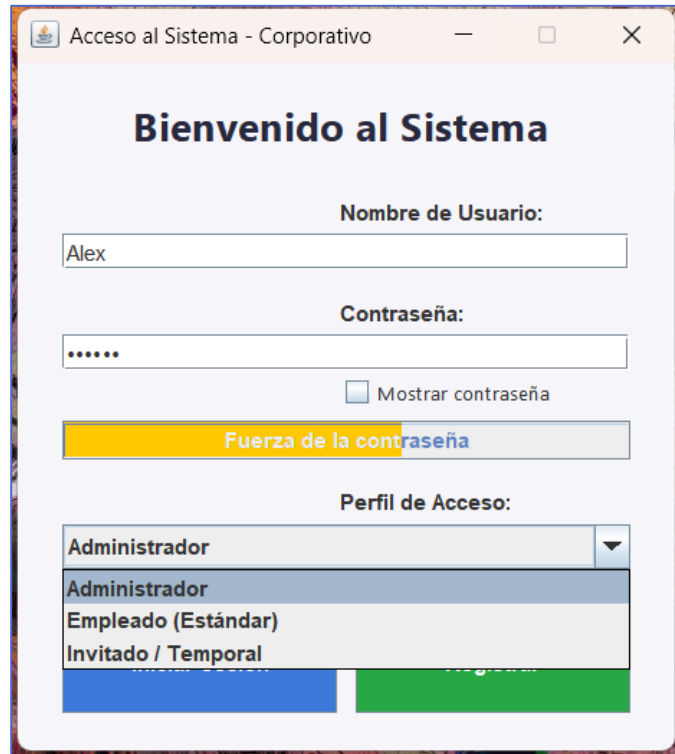
Investigando un poco, nos dimos cuenta de que **Eclipse** estaba **compilando** el código usando **Java 22** o **23**, pero el **JRE** que teníamos **activo** en el sistema era el de **Java 21**. En vez de liarnos a actualizar la máquina de Java, tomamos la decisión técnica de ir a los ajustes y bajar el nivel de compilación (*Compiler compliance level*) a la 21.



Escogimos **forzar Java 21** aposta porque es la **versión LTS (Long Term Support)** actual; así nos aseguramos de que el **.jar** sea súper compatible y no le "pete" al profesor ni a nadie cuando intente integrarlo en sus propios proyectos. Tras hacer ese cambio y reconstruir el proyecto, arrancó a la primera y la ventana se vio impecable, dándonos luz verde para pasar por fin a la fase de empaquetado.



Ahora ya si podemos ejecutar main y comprobar nuestra clase Login.



Acceso al Sistema - Corporativo

Bienvenido al Sistema

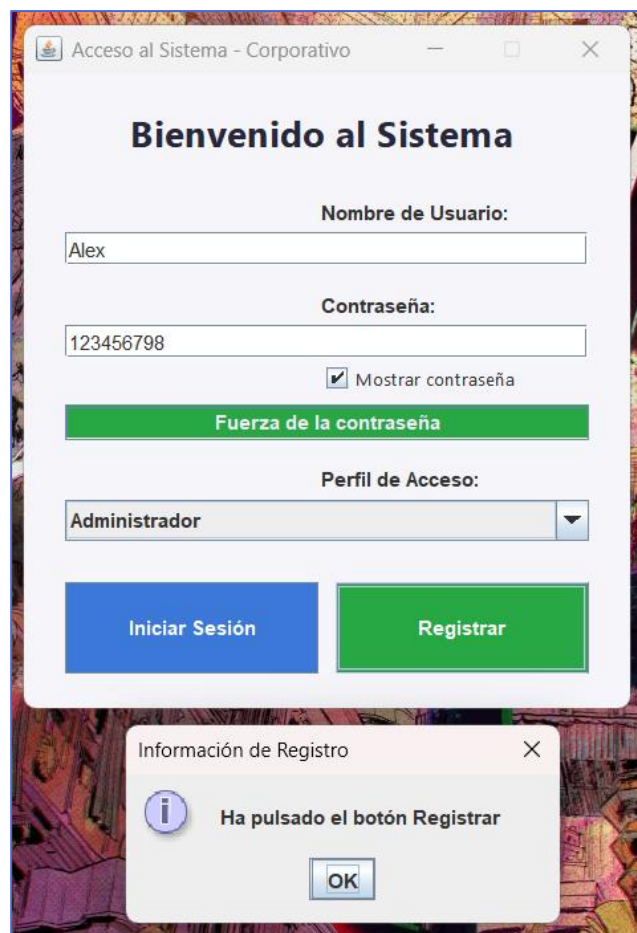
Nombre de Usuario:
Alex

Contraseña:
.....

☐ Mostrar contraseña

Fuerza de la contraseña

Perfil de Acceso:
Administrador
Administrador
Empleado (Estándar)
Invitado / Temporal



Acceso al Sistema - Corporativo

Bienvenido al Sistema

Nombre de Usuario:
Alex

Contraseña:
123456798

☒ Mostrar contraseña

Fuerza de la contraseña

Perfil de Acceso:
Administrador

Iniciar Sesión Registrar

Información de Registro

Ha pulsado el botón Registrar

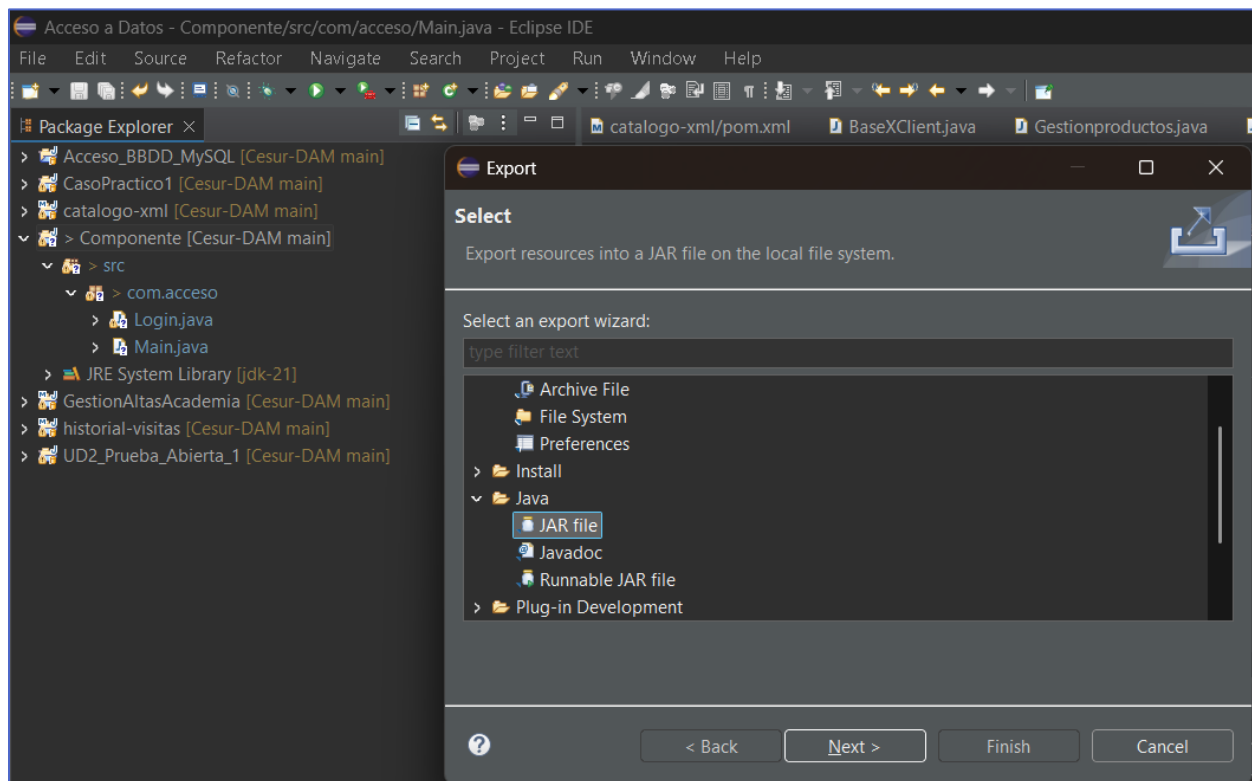
OK

EMPAQUETADO, DISTRIBUCIÓN Y PRUEBAS

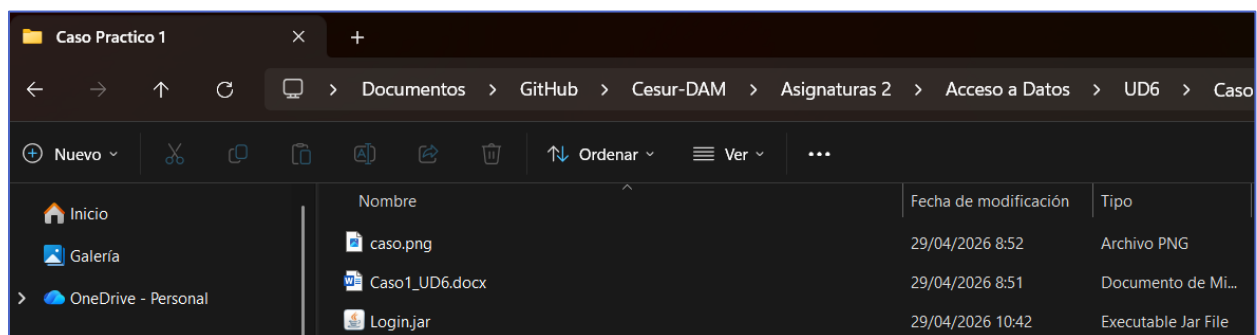
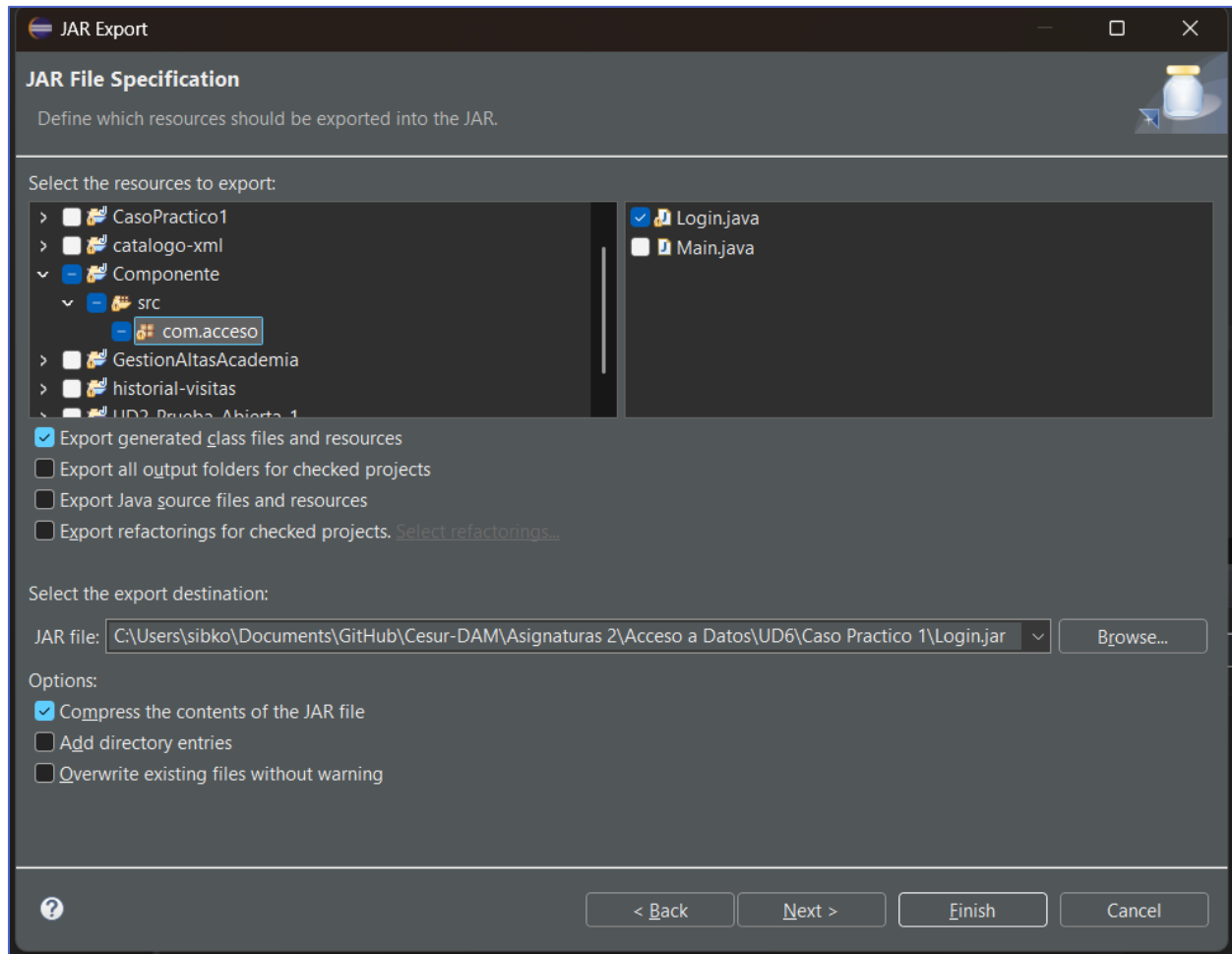
Una de las cosas que más nos ha gustado de desarrollar bajo este paradigma no es solo programar los componentes, sino lo fácil que es luego pasárselos a otros departamentos para que los integren en sus propios proyectos.

2.1 Generación del Archivo .jar

Cuando ya teníamos la clase **Login.java** terminada y bien probada, pasamos a empaquetarla. Usamos el **asistente de exportación** de nuestro IDE (Eclipse).



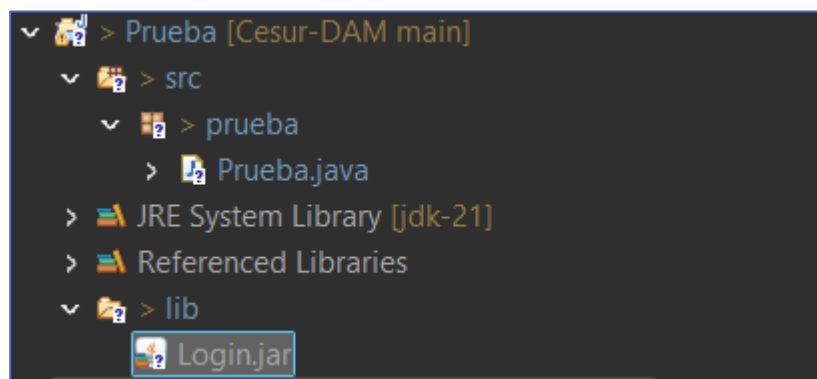
Cogimos todo el **paquete com.acceso** (con sus **archivos** compilados **.class** y sus metadatos **menos la clase Main** que hicimos para probar la app) y lo comprimimos en un **archivo Java Archive** que llamamos **login.jar**. Este archivo encapsula de golpe toda la interfaz gráfica y la lógica de los botones, convirtiéndose en nuestra "pieza de Lego" particular lista para repartir.



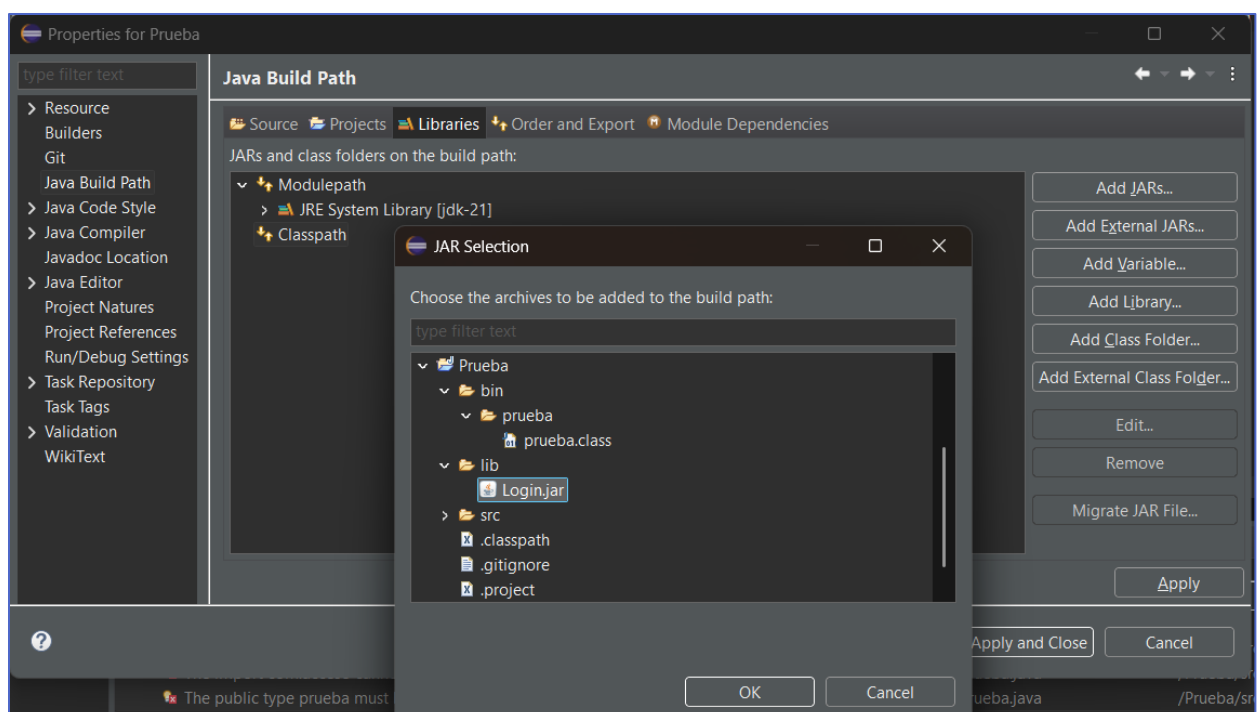
2.2 Prueba de Integración en un Proyecto Independiente

Para comprobar empíricamente que esto de la reutilización funcionaba de verdad, montamos un entorno de pruebas totalmente separado del proyecto original. Los pasos que seguimos fueron súper básicos:

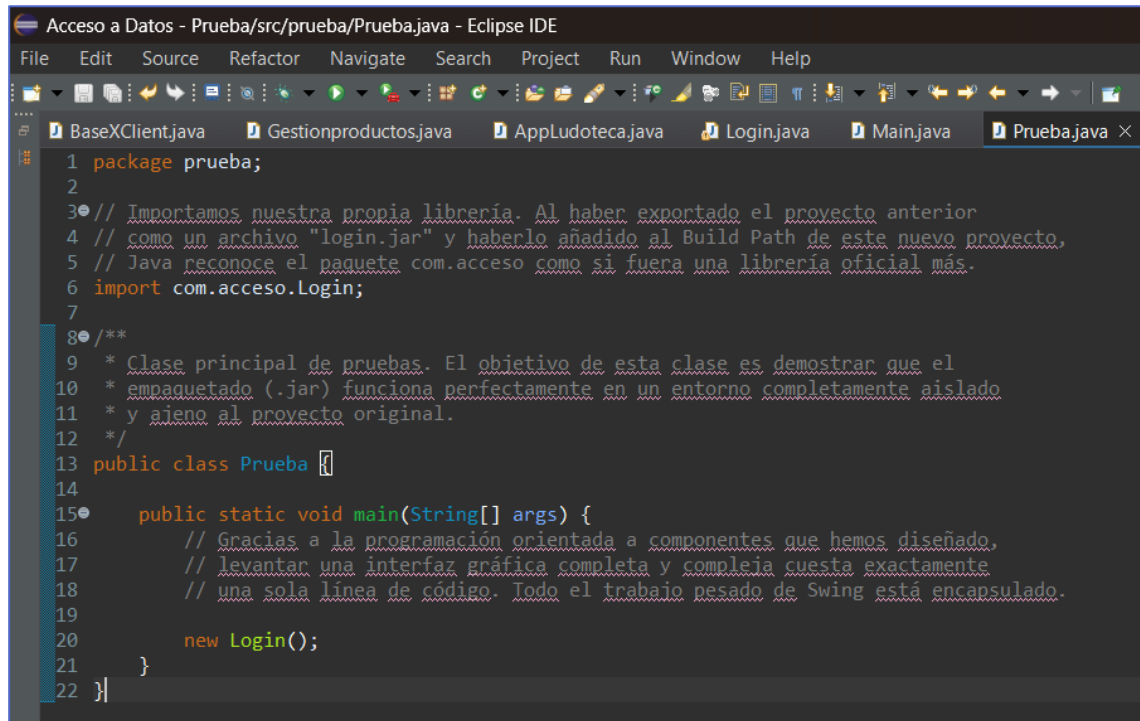
1. Creamos un **proyecto** de Java desde cero, al que llamamos **prueba**.
2. Como queríamos hacer el **proyecto** lo más **portable** posible, creamos una **carpeta lib** en la raíz del proyecto y copiamos allí nuestro **login.jar** creado en el anterior punto.



3. Nos fuimos a la configuración del **Build Path** (ruta de construcción) del proyecto y **añadimos** nuestro archivo **login.jar** como archivo .jar **no externo**. Lo pusimos en **nuestra carpeta lib** del proyecto.

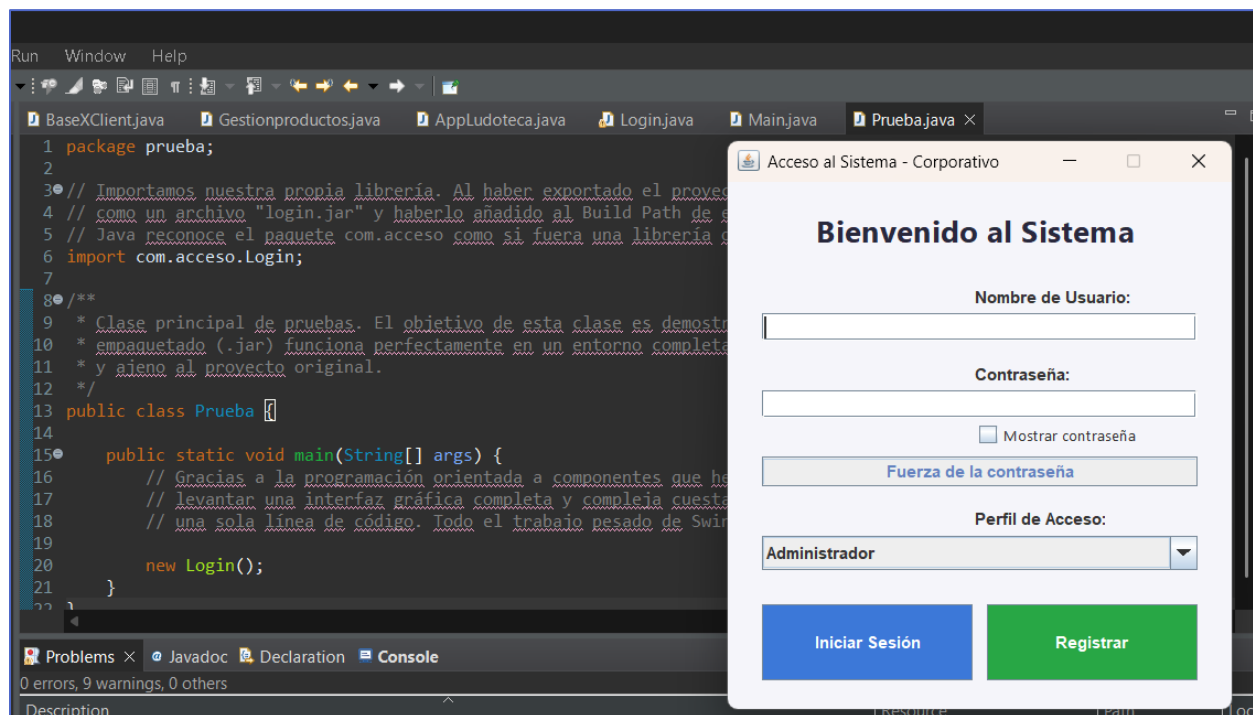


4. Hicimos una **clase principal ejecutora (prueba.java)** con su método **main** de toda la vida.



```
1 package prueba;
2
3 // Importamos nuestra propia librería. Al haber exportado el proyecto anterior
4 // como un archivo "login.jar" y haberlo añadido al Build Path de este nuevo proyecto,
5 // Java reconoce el paquete com.acceso como si fuera una librería oficial más.
6 import com.acceso.Login;
7
8 /**
9  * Clase principal de pruebas. El objetivo de esta clase es demostrar que el
10  * empaquetado (.jar) funciona perfectamente en un entorno completamente aislado
11  * y ajeno al proyecto original.
12  */
13 public class Prueba {
14
15     public static void main(String[] args) {
16         // Gracias a la programación orientada a componentes que hemos diseñado,
17         // levantar una interfaz gráfica completa y compleja cuesta exactamente
18         // una sola línea de código. Todo el trabajo pesado de Swing está encapsulado.
19
20         new Login();
21     }
22 }
```

El momento de la verdad para ver si el paradigma de componentes funcionaba fue el código de esta última clase. Para levantar de golpe toda esa interfaz que nos había costado diseñar, solo tuvimos que **importar el paquete** y llamar a nuestro **método Login()**. Vemos que ejecuta nuestra clase Login importada.



CONCLUSIONES

Desarrollar este caso práctico nos ha venido muy bien para interiorizar que hacer una buena arquitectura de software no va solo de cómo guardamos los datos, sino de aprender a empaquetar nuestra lógica para que el código sea escalable y fácil de mantener a lo largo del tiempo.

El componente de "Acceso al Sistema" cumple de sobra con lo que nos pedía el caso práctico: unifica el diseño usando Java Swing, tiene controles interactivos un poco más avanzados que mejoran el uso diario y, lo más importante, está encapsulado en un .jar que nos va a reducir muchísimo los tiempos de desarrollo de ahora en adelante.

La próxima vez que la empresa arranque un proyecto que necesite control de usuarios, nuestro componente estará ahí, a un triste import de distancia, ahorrándonos tener que reinventar la rueda otra vez.

0 3

CÓDIGO IMPLEMENTADO
(VERSIÓN FINAL)

0 4

EXPLICACIÓN LÍNEA POR
LÍNEA (PRINCIPALES
FRAGMENTOS)

0 6

PRUEBAS Y RESULTADOS



0 6

R E F E R E N C I A S